
Deep Models Compression

Muhammad Safiullah Ibrahim Farrukh Mustafa Abbas

Abstract

This report investigates three principal deep learning model compression techniques—pruning, quantization, and knowledge distillation—aimed at reducing model size and computational demands for resource-limited environments. Experiments conducted on VGG models with CIFAR-100 data assessed each technique’s effects on accuracy, inference time, storage, and hardware compatibility. Pruning was evaluated through both unstructured and structured methods, with structured pruning achieving notable efficiency gains on compatible hardware, while unstructured pruning preserved higher accuracy. Quantization was explored with Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT), where QAT demonstrated improved accuracy retention at low bit-widths, albeit with increased computational demands. Knowledge distillation, through methods such as logit matching and feature alignment, enabled compact student models to closely approximate teacher performance with reduced computational cost, although inheriting certain limitations from the teacher. The results reveal that QAT-based quantization is optimal for memory efficiency, structured pruning excels in real-time hardware deployment, and knowledge distillation offers a balance of accuracy and interpretability, underscoring the distinct strengths of each technique. The code for these experiments can be found in the GitHub repository [here](#).

1. Introduction

With the growing adoption of deep learning models across various applications, reducing their storage and computational cost has become crucial, especially for real-time and resource-constrained environments like mobile devices and embedded systems. In this paper, we have explored three key model compression techniques—pruning, quantization, and knowledge distillation—that aim to reduce model complexity while retaining performance. These techniques address the challenge of deploying deep neural networks on platforms with limited memory, computation power, and energy resources.

Each compression technique was tested through various approaches. In pruning, we implemented both unstructured and structured pruning to identify the relative impact of sparsity at the weight and channel levels. For quantization, we investigated Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT) across different bit-widths to study the trade-off between model precision and memory savings. Finally, in knowledge distillation, we applied multiple distillation strategies, such as logit matching and feature representation matching, to determine how effectively a student model can approximate the performance of a larger teacher model.

This paper provides a hands-on experience with model compression strategies, allowing for a comprehensive understanding of the trade-offs and implementation challenges associated with each technique. Through detailed experimentation and analysis, we gained insights into how each method affects model performance, storage requirements, and computational efficiency, setting the groundwork for their application in real-world scenarios.

2. Methodology

2.1. Dataset Preparation and Model Setup

In all tasks, we employed the CIFAR-100 dataset, which comprises 60,000 color images categorized into 100 classes, with 600 images per class. The following preprocessing steps were applied:

Images were resized to 224×224 pixels to align with the input dimensions expected by the VGG-11 model. Each image was normalized using the mean and standard deviation specific to the CIFAR-100 dataset. A batch size of 64 was utilized for efficient data loading during both training and testing.

For the model setup, we utilized the pre-trained VGG-11 architecture for task 1, 2 and VGG-16 architecture for task 3 from the `torchvision.models` library. Adjustments were made to tailor the model for the CIFAR-100 classification task. The final fully connected layer was replaced to output 100 classes instead of the original 1000. Cross-Entropy Loss was used as the objective function. AdamW optimizer with a learning rate of 1×10^{-4} . The model was trained for 3 epochs.

2.2. Pruning

2.2.1. UNSTRUCTURED AND STRUCTURED PRUNING

Weight Tensor Distribution Visualization

To understand the effects of pruning on the VGG-11 model’s internal parameters, we visualized the weight distributions of each layer before and after applying both unstructured and structured pruning methods. The objective was to assess how each pruning strategy affects weight distribution and parameter sparsity across layers, providing insights into the internal adjustments the model undergoes during pruning.

For unstructured pruning, individual weights with the smallest magnitudes (closest to zero in absolute terms) were removed, as they contribute least to model performance. A global sparsity level of 90% was applied, meaning 90% of the weights were pruned across all layers to emphasize the effects on weight distribution. The procedure involved collecting weights from all convolutional and fully connected layers, plotting histograms of these weights before pruning to capture the initial distribution, and setting a threshold to retain only the largest 10% of weights. A binary mask was then applied to prune weights below this threshold, setting them to zero.

In contrast, structured pruning removes entire channels (filters) in convolutional layers based on their importance, preserving the structural integrity of the network and facilitating efficient computation. The L2 norm of the filter weights was used as a measure of importance, with channels having lower L2 norms identified as less significant and targeted for removal. For each layer, the function `compute_channel_importance` calculated these norms, offering a quantitative basis for determining channel relevance. The function `prune_conv_layer_classifier_fix` (Appendix B) was then used to remove the least important channels, while adjusting subsequent convolutional and fully connected layers to maintain input consistency across the network. This structured approach effectively reduces the number of parameters in a way that is compatible with hardware optimizations, enabling potential storage and inference-time reductions. Structured pruning, however, maintains a bell-curve distribution in weight histograms, with a reduction in the frequency of values instead of creating sparse distributions, as observed in unstructured pruning.

Sensitivity Analysis

A sensitivity analysis was conducted to assess the impact of pruning on each individual layer of the VGG-11 model using both unstructured and structured pruning methods. The goal was to identify layers that are more tolerant to pruning and those that are essential for maintaining model performance, providing guidance for setting sparsity levels

in the final sparsity induction phase.

For unstructured pruning, each convolutional and fully connected layer in the model was examined independently at various sparsity ratios, specifically 10%, 30%, 50%, 70%, and 90%. For each layer and sparsity level, a copy of the model was created where only the targeted layer was pruned, while other layers remained intact. The model was then evaluated on the test set to measure the impact on accuracy. The results highlighted the relative sensitivity of each layer to sparsity, as reflected in the accuracy versus sparsity ratio plots. A crucial component of this process was ensuring that pruned weights remained zero during subsequent fine-tuning. To achieve this, backward hooks were incorporated into the `prune_model_per_layer` function (Appendix A), which modifies gradient updates to ensure that pruned weights do not receive gradient-based updates. This approach maintains the sparsity levels throughout fine-tuning, preserving the efficiency gains from pruning.

For structured pruning, a similar analysis was performed, focusing solely on the convolutional layers. Here, entire channels (filters) were targeted for pruning rather than individual weights, with the L2 norm of the filter weights used as a measure of each channel’s importance. Channels with lower L2 norms were identified as less significant and thus were prioritized for removal. The `prune_conv_layer_classifier_fix` function (Appendix B) was employed to remove the specified channels and adjust subsequent layers to maintain structural consistency. After pruning each layer, the model’s accuracy was evaluated on the test set, and this process was systematically repeated across the same range of sparsity ratios (10%, 30%, 50%, 70%, and 90%). This evaluation allowed us to observe how pruning entire channels affects each layer’s contribution to model accuracy, with results informing optimal sparsity settings for each convolutional layer in the final model configuration.

Final Sparsity Induction

Using insights gained from the sensitivity analysis, a structured approach was adopted to achieve an overall sparsity of approximately 25% across the VGG-11 model. This approach aimed to balance model compression with minimal impact on accuracy. The final sparsity induction process involved layer-specific sparsity allocation, pruning, fine-tuning, and evaluation to ensure that performance was maintained at acceptable levels post-pruning.

Layer-wise sparsity ratios were assigned based on each layer’s sensitivity to pruning, as observed in the sensitivity analysis. This allocation ensured that critical layers with high sensitivity were either left unpruned or pruned conservatively, while layers with greater tolerance were pruned more aggressively. For unstructured pruning, early convo-

lutional layers (`features.0`) were assigned a sparsity of 0%, preserving their critical role in low-level feature extraction. The remaining convolutional layers were assigned a higher sparsity of 45% to reduce parameters without significantly impacting model performance. Fully connected layers (`classifier.0` and `classifier.3`) received a sparsity of 24%, reflecting their moderate sensitivity, while the output layer (`classifier.6`) was kept at 0% sparsity to preserve classification accuracy, given its essential role in final class predictions.

For structured pruning, sparsity was allocated across convolutional layers as follows:

- **Early Convolutional Layers** (`features.0`, `features.3`): Assigned 0% sparsity to maintain low-level feature integrity.
- **Middle Convolutional Layers** (`features.13`, `features.11`, `features.6`): Set to 20% sparsity, balancing parameter reduction with moderate accuracy retention.
- **Deeper Convolutional Layers** (`features.16`, `features.18`): Assigned 28% sparsity, as these layers demonstrated higher tolerance to pruning.

Once the appropriate sparsity levels were determined, the pruning functions `prune_model_per_layer` (Appendix A) and `prune_conv_layer_classifier_fix` function (Appendix B) was applied to induce the specified sparsity ratios within each layer. Fine-tuning was conducted for an additional 3 epochs using the original training configuration to allow the model to adjust to the new parameter structure, helping to mitigate any performance loss from pruning.

After fine-tuning, the pruned models were evaluated on the test set to confirm their performance. Both unstructured and structured pruning maintained model accuracy close to the baseline while achieving the desired 25% sparsity.

2.2.2. COMPARISON AND ANALYSIS

Grad-CAM Visualizations

To evaluate the robustness of the pruning methods in terms of feature localization and accuracy retention, Grad-CAM (Gradient-weighted Class Activation Mapping) visualizations were generated for the original, unstructured-pruned, and structured-pruned VGG-11 models. Grad-CAM (Selvaraju et al., 2019) provides insight into which regions of an input image are most influential in the model’s decision-making process by highlighting the areas that activate the highest gradients. This qualitative analysis helps in understanding how pruning affects the model’s ability to focus on relevant features for classification.

Three versions of the VGG-11 model were loaded: Baseline, Unstructured and Structured model. An input image was selected and preprocessed to match the input requirements of the VGG-11 model. This involved resizing the image to 224×224 pixels, converting it to a tensor, and normalizing it using the CIFAR-100 dataset’s mean and standard deviation values.

For each model, Grad-CAM was applied to the preprocessed image to generate heatmaps that highlight the regions of the image most relevant to the model’s predictions. The process involved identifying the target convolutional layer within the model to extract feature maps, which was the last convolutional layer in all three cases as it learns complex patterns. The gradients of the target class score were computed with respect to the feature maps, and a weighted combination of these gradients was used to produce the activation map. Finally, the activation map was overlaid onto the original image to visualize the areas of interest.

The resulting Grad-CAM heatmaps were visualized alongside the original image to compare how each model attends to different regions. This comparison highlighted the impact of pruning on the model’s focus areas, revealing whether pruned models maintain coherent feature localization similar to the baseline model.

The implementation of the Grad-CAM visualizations can be found in Appendix C.

Inference Time and Storage Comparison

To evaluate the practical implications of the pruning methods, inference times and on-device storage requirements were measured for the original, unstructured-pruned, and structured-pruned VGG-11 models. These metrics provide insights into the efficiency gains and resource savings achieved through pruning.

The average inference time for a single query image was measured over 100 runs for each model variant (original, unstructured-pruned, and structured-pruned). A dummy input tensor of size $1 \times 3 \times 224 \times 224$ was used to simulate the input. The total number of parameters in each model was calculated to estimate the storage size. Each parameter was assumed to occupy 4 bytes (32 bits).

Accuracy vs. Target Sparsity

To quantitatively evaluate the trade-off between model sparsity and accuracy, an accuracy versus target sparsity plot was generated for both unstructured and structured pruning methods. This analysis provides a clear visualization of how increasing sparsity levels affect the model’s performance.

Five target sparsity levels were selected: 0%, 20%, 40%, 60%, and 80%. For each sparsity level, both unstructured and structured pruning were applied to the model. After

pruning, the models were evaluated on the test set to record their accuracies. The recorded accuracies were plotted against the corresponding sparsity levels for both pruning methods, allowing for a direct comparison of their performance impacts.

2.3. Quantization

Preparation of the dataset used and baseline model have already been defined in Section 2.1

2.3.1. POST TRAINING QUANTIZATION:

For Post Training Quantization (PTQ), we apply quantization directly to the trained model, casting both the model and the inputs to the required data type for each quantization scheme. PyTorch’s native casting functions were used for float16 (fp16) and bfloat16 (bf16) data types. For the integer-based quantization schemes, specifically int8 and int4, we leveraged the `quantize_` API from torchao.

In the case of int8 and int4 quantizations, floating-point values are mapped to integer values through a linear transformation, which includes computing a scale factor s and a zero-point z . These parameters enable the conversion of each floating-point value x to an integer q via the formula:

$$q = \text{round} \left(\frac{x}{s} \right) + z$$

This was handled by torchao’s API.

More specifically, we used the `int8_dynamic_activation_int8_weight()` scheme for int8 and `int4_weight_only()` for int4 dtype (due to the absence of an API which casts the activations to int4 as well). int4 also has a higher risk of reducing accuracy due to the limited range of representable values. To control this impact, we used a weight-only approach. To this end, we kept running into Cuda compatibility errors on Colab and Kaggle as well, which were eventually resolved by running the said code on a 4090 GPU..

2.3.2. QUANTIZATION AWARE TRAINING

For QAT, the float16 and bfloat16 quantizations were handled natively in PyTorch by casting the model and input data to the respective data types. These conversions were followed by a standard training loop, which allowed the model to adjust and recover from any accuracy losses due to quantization. However, it was observed that float16 quantization often led to unstable training due to the lower precision, which resulted in the loss function quickly converging to NaN values. To address this instability, we switched from the Adam optimizer to Stochastic Gradient Descent (SGD), which is more stable in low-precision training settings. Furthermore, to avoid overfitting, especially given that the baseline model was already well-trained, we adopted a smaller

learning rate.

Moreover, for int8, QAT was implemented using `Int8DynActInt4WeightQATQuantizer()` from torchao, which applies fake quantization to linear layers and simulates quantization effects without actual dtype casting. The model was then converted using `quantizer.convert()` post-finetuning. Similarly, for int4 QAT, we used the `Int4WeightOnlyQATQuantizer()` from torchao. Given that int4 quantization does not have native support for activating both weights and activations to int4, fake quantization was applied only to the weights, with the model being cast to bfloat16 prior to training.

Memory footprints for each model were calculated by iterating through each parameter and summing the product of the parameter count and its bit-width. This enabled a direct comparison of memory savings achieved by each quantization approach. It is to be noted that the model sizes do not contain the size of scaling factor s and zero point c , which is much smaller than the size of layers themselves.

2.4. Knowledge Distillation

Knowledge Distillation (KD) is an advanced technique aimed at compressing deep learning models by transferring knowledge from a large, complex model (teacher) to a smaller, simpler model (student). KD enables the student model to approximate the behavior of the teacher, with the primary goal of retaining the predictive power and knowledge representation of the larger model while reducing computational resources and storage requirements. The KD process typically involves "softened" outputs from the teacher model, which carry richer information than standard hard labels. These outputs, or softened logits, enable the student to learn more effectively from the teacher’s nuanced predictions. Over time, KD has evolved to include methods beyond simple logit matching, such as Hint-based Distillation, which transfers intermediate feature representations, and Contrastive Representation Distillation, which emphasizes alignment between the teacher’s and student’s learned representations.

2.4.1. MODEL CONFIGURATIONS AND TRAINING SETUP

In this series of experiments, we employed a VGG-16 model as the teacher network, configured specifically for the CIFAR-100 dataset. The data preprocessing pipeline involved resizing all CIFAR-100 images to 224×224 pixels, compatible with the input dimensions required by the VGG-16 architecture. Following resizing, we applied normalization with a mean and standard deviation of (0.5, 0.5, 0.5). A batch size of 64 was utilized for efficient data loading during both training and testing.

During the fine-tuning phase, we used an ADAMW optimizer with a learning rate of 0.0001. Fine-tuning was conducted over three epochs for teacher and two epochs for all students (Less epochs due to computational constraints).

To distill knowledge from the VGG-16 teacher model to a more compact student model, we utilized a loss function that combined both Kullback-Leibler (KL) divergence and standard classification loss (Cross Entropy Loss). We set the weight of each component to $\alpha = 0.5$, balancing learning from the softened teacher logits with direct learning from hard class labels. This approach enabled the student model to capture not only the teacher's predictions but also the true label distributions, providing a comprehensive distillation process that captures both high-level behavior and explicit classification accuracy.

2.4.2. GRADIENT OF THE KL DIVERGENCE LOSS WITH RESPECT TO STUDENT LOGIT z_i

Definitions and Setup

Logits:

- z_i : The i -th logit (pre-softmax output) of the student model.
- v_i : The i -th logit (pre-softmax output) of the teacher model.

Soft Probabilities: After applying softmax with temperature T , the student and teacher logits are transformed into probabilities p_i and q_i as follows:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad \text{for the student model,}$$

$$q_i = \frac{\exp(v_i/T)}{\sum_j \exp(v_j/T)} \quad \text{for the teacher model.}$$

Temperature T : The temperature parameter T controls the “softness” of the probabilities. Higher T values make the probabilities closer to a uniform distribution.

KL Divergence Loss for Logit Matching: The KL divergence loss L_{KL} between the teacher and student models is given by:

$$L_{KL} = \sum_i q_i \log \frac{q_i}{p_i}.$$

Step 1: Expand the KL Divergence Loss L_{KL}

The KL divergence can be rewritten as:

$$L_{KL} = \sum_i q_i \log q_i - \sum_i q_i \log p_i.$$

The first term $\sum_i q_i \log q_i$ does not depend on the student logits z_i , so we can ignore it when calculating the derivative with respect to z_i . Thus, we only need to focus on:

$$L_{KL} = - \sum_i q_i \log p_i.$$

Step 2: Substitute for p_i Using the Softmax Function

Since

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)},$$

we can write:

$$\log p_i = \frac{z_i}{T} - \log \left(\sum_j \exp(z_j/T) \right).$$

Then,

$$L_{KL} = - \sum_i q_i \left(\frac{z_i}{T} - \log \left(\sum_j \exp(z_j/T) \right) \right),$$

$$L_{KL} = - \sum_i \frac{q_i z_i}{T} + \sum_i q_i \log \left(\sum_j \exp(z_j/T) \right).$$

Step 3: Differentiate L_{KL} with Respect to z_i

Now, we calculate $\frac{\partial L_{KL}}{\partial z_i}$.

Differentiate the First Term The first term is:

$$- \sum_i \frac{q_i z_i}{T}.$$

Differentiating this term with respect to z_i :

$$\frac{\partial}{\partial z_i} \left(- \sum_j \frac{q_j z_j}{T} \right) = - \frac{q_i}{T}.$$

Differentiate the Second Term The second term is:

$$\sum_i q_i \log \left(\sum_j \exp(z_j/T) \right).$$

Differentiating this with respect to z_i requires the chain rule:

$$\begin{aligned} & \frac{\partial}{\partial z_i} \left(\sum_j q_j \log \left(\sum_k \exp(z_k/T) \right) \right) \\ &= \sum_j q_j \cdot \frac{1}{\sum_k \exp(z_k/T)} \cdot \frac{\partial}{\partial z_i} \left(\sum_k \exp(z_k/T) \right) \\ &= \sum_j q_j \cdot \frac{1}{\sum_k \exp(z_k/T)} \cdot \frac{\exp(z_i/T)}{T} \\ &= \frac{\exp(z_i/T)}{T \sum_k \exp(z_k/T)} \sum_j q_j = \frac{p_i}{T}. \quad (1) \end{aligned}$$

Step 4: Combine Terms

Now, putting together the terms we derived:

$$\frac{\partial L_{KL}}{\partial z_i} = -\frac{q_i}{T} + \frac{p_i}{T} = \frac{p_i - q_i}{T}.$$

Step 5: Approximate Using z_i and v_i

To approximate the difference $p_i - q_i$ for small differences between the student logits z_i and teacher logits v_i , we use a first-order Taylor expansion. This approximation assumes that the logits z_i are close to v_i , so p_i and q_i are also close.

Taylor Expansion for Small Differences: Since p_i and q_i are derived from the softmax function, we can express p_i as a function of z_i and expand it around v_i . For small values of $(z_i - v_i)$, the softmax function $p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$ can be approximated using a first-order Taylor expansion in terms of $q_i = \frac{\exp(v_i/T)}{\sum_j \exp(v_j/T)}$.

First-Order Approximation: We express p_i in terms of q_i and the difference $(z_i - v_i)$ as follows:

$$p_i \approx q_i + \frac{\partial p_i}{\partial z_i} (z_i - v_i).$$

To compute $\frac{\partial p_i}{\partial z_i}$, we recall that for the softmax function p_i , the derivative with respect to z_i is given by:

$$\frac{\partial p_i}{\partial z_i} = \frac{1}{T} p_i (1 - p_i).$$

Since $p_i \approx q_i$, we substitute $p_i \approx q_i$ into the derivative:

$$\frac{\partial p_i}{\partial z_i} \approx \frac{q_i (1 - q_i)}{T}.$$

Thus, we can approximate p_i as:

$$p_i \approx q_i + \frac{q_i (1 - q_i)}{T} (z_i - v_i).$$

Final Approximation of $p_i - q_i$: Now, substituting this back, we obtain:

$$p_i - q_i \approx \frac{q_i (1 - q_i)}{T} (z_i - v_i).$$

Therefore, for small logit differences, the difference $p_i - q_i$ is approximately proportional to $(z_i - v_i)$, scaled by the term $\frac{q_i (1 - q_i)}{T}$. This provides a linear approximation to $p_i - q_i$, which can simplify calculations when applying gradient-based optimization methods.

Now, substituting this back, we obtain:

$$p_i - q_i \approx \frac{q_i (1 - q_i)}{T} (z_i - v_i).$$

Thus, the difference $p_i - q_i$ is approximately proportional to $(z_i - v_i)$, scaled by $\frac{q_i (1 - q_i)}{T}$, for small differences between logits.

For sufficiently small q_i and T , we can further approximate this as:

$$p_i - q_i \approx \frac{1}{NT^2} (z_i - v_i).$$

2.4.3. ASPECTS OF LOGIT MATCHING

Logit Matching provides a foundational approach for knowledge distillation by enabling the student model to mimic the teacher's output patterns directly. This approach leverages a temperature scaling parameter T to soften both sets of logits, allowing for more nuanced comparison and distillation. The main objective here is to minimize the Kullback-Leibler (KL) divergence between the teacher's softened logits and those of the student. This divergence, scaled by T^2 , ensures that the softened logits can effectively guide the student model's training.

Basic Logit Matching

For Logit Matching, we minimize the Kullback-Leibler (KL) divergence between the softened logits of the student and teacher. The softened logits are obtained using temperature scaling, which controls the smoothness of the output probabilities. The loss function for Logit Matching is defined as:

$$\mathcal{L}_{\text{logit}} = \text{KL} \left(\text{Softmax} \left(\frac{\text{student_logits}}{T} \right), \text{Softmax} \left(\frac{\text{teacher_logits}}{T} \right) \right) \quad (2)$$

where T is the temperature parameter that controls the smoothness of the probability distribution.

Label Smoothing Loss

The Label Smoothing Loss aims to prevent the student model from becoming overconfident in its predictions. This is achieved by distributing a small probability mass across incorrect classes, instead of assigning all probability mass to the target class.

Mathematically, this is done by defining a confidence level $\alpha = 1 - \text{smoothing}$, assigned to the correct class, while the remaining probability mass is distributed across the other classes. Label Smoothing Loss, $\mathcal{L}_{\text{LS}}$, is then computed based on the cross-entropy between the smoothed labels and model predictions, with smoothing parameter s .

Decoupled Knowledge Distillation (DKD) Loss

The Decoupled Knowledge Distillation (DKD) Loss refines the distillation process by separating target and non-target classes. For target classes, the objective is to minimize the mean squared error (MSE) between the student and

teacher logits, focusing on exact value matching. For non-target classes, the KL divergence is used to capture the distributional similarity between the student and teacher logits, also scaled by T^2 .

To achieve this, a target mask is created to separate target and non-target classes within the logits. The DKD Loss, $\mathcal{L}_{\text{DKD}}$, combines these separate loss components for both target and non-target classes, with an added regularization term to prevent divergence:

$$\mathcal{L}_{\text{DKD}} = \mathcal{L}_{\text{target}} + \mathcal{L}_{\text{non-target}} + \lambda (\|\text{student_logits}\| + \|\text{teacher_logits}\|) \quad (3)$$

where $\mathcal{L}_{\text{target}}$ represents the MSE for target classes and $\mathcal{L}_{\text{non-target}}$ is the KL divergence for non-target classes. The regularization term, scaled by λ , controls for overfitting and ensures model stability.

2.4.4. COMPARING PERFORMANCE OF SOTA APPROACHES

To evaluate the effectiveness of different Knowledge Distillation (KD) techniques, we implement and compare several approaches, including Logit Matching, Hint-based Distillation, and Contrastive Representation Distillation. These techniques aim to transfer knowledge from the teacher model to the student model, improving the student's performance on the CIFAR-100 dataset. In our experiments, we used four different student models:

- Independent Student Model (baseline without KD)
- Student Model with Logit Matching
- Student Model with Hint-based Distillation
- Student Model with Contrastive Representation Distillation (CRD)

Each of the student models is based on the VGG-11 architecture with an adjusted final layer to classify CIFAR-100's 100 classes. The loss function for each approach is as follows:

Hint-based Distillation

For Hint-based Distillation, we compute the Mean Squared Error (MSE) loss between the feature maps of the teacher and student models. The hint loss encourages the student to match the intermediate feature representations of the teacher:

$$\mathcal{L}_{\text{hint}} = \frac{1}{N} \sum_{i=1}^N \|\text{student_features}_i - \text{teacher_features}_i\|^2$$

where N is the number of elements in the feature map, and the student and teacher features are taken from intermediate layers.

Contrastive Representation Distillation (CRD)

In Contrastive Representation Distillation, the objective is to minimize the distance between the feature representations of corresponding student-teacher pairs (positive pairs) while maximizing the distance between non-corresponding pairs (negative pairs). This approach encourages the student to align its features closely with the teacher's while maintaining a minimum separation from unrelated features.

The CRD loss consists of two main components: the positive pair loss, which encourages alignment between student and teacher features, and the negative pair loss, which enforces separation. The loss function is defined as:

$$\mathcal{L}_{\text{crd}} = -\log(\sigma(\text{student_features} \cdot \text{teacher_features})) - \log(1 - \sigma(\text{margin} - \text{student_features} \cdot \text{teacher_features})) \quad (4)$$

where σ denotes the sigmoid function, and the margin sets a minimum distance for non-target pairs.

The overall loss function for training the student model combines the distillation loss with a classification loss, weighted by the hyperparameter α . We kept the $\alpha = 0.5$ so equal importance is given to both weights.

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{KD}} + (1 - \alpha) \cdot \mathcal{L}_{\text{classification}}$$

where $\mathcal{L}_{\text{KD}}$ can represent any of the distillation methods (Logit Matching, Hint-based Distillation, or CRD), and $\mathcal{L}_{\text{classification}}$ is the cross-entropy loss for the main classification task.

2.4.5. COMPARING PROBABILITY DISTRIBUTIONS

Once the student models were trained, an analysis was performed to compare the probability distributions of the teacher's predictions with those of each student model. The goal was to quantify how closely each student model approximated the teacher's softened probability outputs. This was done by computing the Kullback-Leibler (KL) divergence to measure the discrepancy between the teacher's and student's probability distributions. To ensure consistency, the logits of both the teacher and student models were softened using a temperature scaling factor T before calculating the KL divergence.

We computed it on 3 of query images randomly sampled by comparing the outputs of the teacher and student models. This allowed us to assess how well each student model approximated the teacher's softened probability distributions. The student model with the lowest KL divergence was considered to have the most closely aligned distribution with the teacher. It also allowed for a comparison of the KD-based student models with the independent student to determine which model's distribution was more closely aligned with the teacher's predictions.

2.4.6. EXAMINING LOCALIZATION KNOWLEDGE TRANSFER

In addition to label prediction, a key advantage of Knowledge Distillation (KD) is transferring the teacher model’s ability to localize important features in an image. We tested this by using Grad-CAM visualizations to investigate whether students trained with KD methods could focus on the same relevant image areas as the teacher. We aimed to assess if KD improved the student model’s interpretability by comparing the attention maps of the teacher and student models.

To generate Grad-CAM visualizations, we used the following procedure. First, we identified the last convolutional layer of the model using the function ‘find last conv layer’. Then, we captured gradients and activations using hooks in the ‘generate gradcam’ function. Gradients were pooled across channels, and the activations were weighted by these pooled gradients to generate heatmaps. We applied this process to both teacher and student models.

The Grad-CAM visualizations were then compared across models by evaluating the degree of overlap between their attention maps. For each model, we computed the Intersection over Union (IoU) between the teacher’s and student’s heatmaps using the ‘calculate iou’ function.

This allowed us to examine whether the student model could localize features in a manner similar to the teacher model, as indicated by the similarity in their attention maps. The findings from this analysis helped determine if the KD process improved the student’s ability to focus on relevant regions, enhancing interpretability.

2.4.7. CHECKING FOR COLOR INVARIANCE WITH CRD

Invariance to transformations, such as color variations, is an important property for models to learn. To investigate whether the student model could learn color invariance using Contrastive Representation Distillation (CRD), we applied color jitter augmentation during the training phase. This augmentation included variations in brightness, contrast, saturation, and hue, with values set to 0.4, 0.4, 0.4, and 0.2, respectively.

The CIFAR-100 dataset was loaded with the color jitter transformations applied, allowing the model to learn from color-jittered inputs. The teacher model, T, was finetuned with color jitter applied to the training data. We then assessed whether the student model could handle color variations during inference. An example image is shown in figure 1,

Then distillation process was carried out using the color-invariant teacher model and regular augmentations applied to the student model. Then we evaluated the performance

of the student model on a color-jittered version of the validation set to determine if it had learned to ignore color variations. The goal was not to explicitly make the student model invariant to color but to investigate whether the properties of the color-invariant teacher could be distilled ambiently into the student. We compared these results with the independent student to see whether the teacher passed the colour inavriance to the student model.

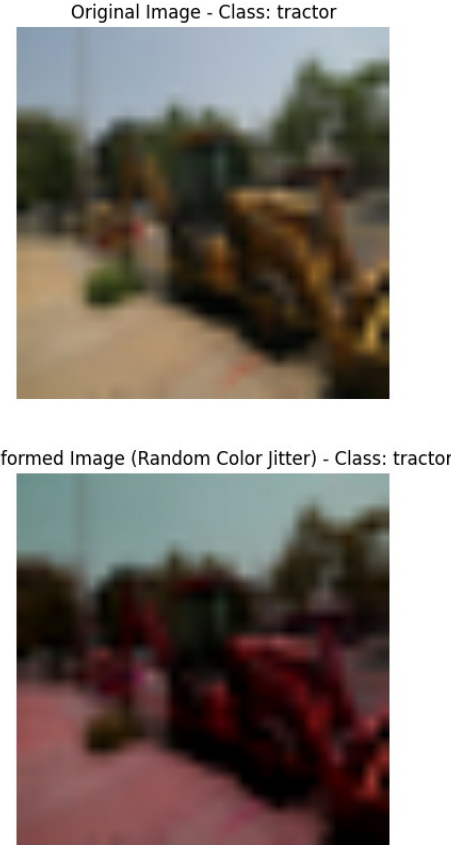


Figure 1. Image with colour jitter applied

2.4.8. TESTING THE EFFICACY OF A LARGER TEACHER

To conclude, we further evaluated how the teacher’s size influences KD performance. Using Logit Matching, we compared the VGG-16 and VGG-19 teacher models to observe if a larger teacher model yields better results for the student in terms of accuracy and other relevant metrics. This analysis explored whether there is a significant benefit to using a larger teacher, or if smaller teachers are sufficient for effective knowledge transfer.

3. Results

3.1. Pruning

3.1.1. UNSTRUCTURED AND STRUCTURED PRUNING

Weight Distribution of Tensors

The weight distribution before and after pruning was analyzed for both unstructured and structured pruning methods across all convolutional and fully connected layers in the VGG-11 model. This analysis provides insights into the impact of pruning on the model’s weights, where less significant weights are removed. For unstructured pruning, weights with smaller magnitudes were removed across all layers, whereas structured pruning targeted entire channels based on their computed importance.

As seen in Figure 2, unstructured pruning results in a large spike of weights at zero for most layers, indicating a high level of sparsity induced by applying 90% sparsity to each layer. This aggressive pruning strategy produces many zero-valued weights while retaining important non-zero weights.

In contrast, structured pruning (Figure 3) preserves the bell-shaped distribution of weights across layers but reduces the frequency of different weight values by removing entire channels. With a target sparsity of 25% applied uniformly to all convolutional layers, the structured pruning process maintains more of the original weight distribution shape while effectively reducing the model’s overall parameter count.

Sensitivity Analysis of Layers Against Sparsity Ratios

A sensitivity analysis was performed to evaluate the effect of various sparsity ratios on the accuracy of individual layers. Each layer was pruned at different sparsity levels (10%, 30%, 50%, 70%, and 90%) to assess its sensitivity. The post-pruning accuracy for each layer at these levels provides valuable insights into which layers are more robust to pruning and which experience a more significant performance drop, aiding in the decision-making for layer-specific sparsity settings.

In Figure 4, the unstructured pruning analysis reveals that at a sparsity level of 70%, the most affected layers are the initial convolutional layers (Convolutional layers 1, 2, and 4). These layers are critical for learning low-level features, and inducing sparsity in their weights impairs the model’s ability to capture fundamental patterns in the data, leading to a notable drop in accuracy. In contrast, the later convolutional and fully connected (classifier) layers show less sensitivity to high sparsity ratios, indicating that the model can maintain performance with higher sparsity in these layers.

Figure 5 shows the sensitivity analysis results for structured pruning. Here, a 50% sparsity level causes a noticeable accuracy drop in earlier convolutional layers, while later

layers maintain relatively stable performance. Since structured pruning removes entire channels, pruning early layers reduces the model’s capacity to learn essential low-level features. Consequently, later layers, which capture more abstract features, are less affected by higher sparsity levels, supporting the hypothesis that low-level features are more crucial to model performance than high-level features.

Induction of Overall 25% Sparsity and Evaluation

Following the sensitivity analysis, an overall sparsity of approximately 25% was induced across the VGG-11 models for both unstructured and structured pruning approaches. The accuracy of each model was evaluated before and after fine-tuning to observe the impact of this pruning level. Fine-tuning is essential for recovering some of the lost performance, as it allows the remaining non-zero weights to adjust and learn improved representations.

In the unstructured pruning model, higher sparsity was induced in the fully connected layers (classifier layers 1 and 2), as the sensitivity analysis indicated a relatively low accuracy drop when pruning these layers more aggressively. Notably, no sparsity was induced in the first convolutional layer, as pruning this layer heavily impacts accuracy, given its importance in learning low-level features.

For structured pruning, higher sparsity was applied to the last two convolutional layers, as these layers demonstrated resilience to increased sparsity in the sensitivity analysis. Structured pruning targets entire channels instead of individual weights, leading to a more significant initial drop in accuracy.

Table 1 summarizes the key metrics for both unstructured and structured pruning methods, including total parameters, non-zero parameters, zero parameters, overall sparsity, and the model’s accuracy before and after fine-tuning. For unstructured pruning, the accuracy before fine-tuning was 70.64%, which improved to 71.69% after fine-tuning. Structured pruning, however, had a more pronounced drop in accuracy, with a pre-fine-tuning accuracy of 44.28%, which improved to 69.24% after fine-tuning.

Metric	Unstructured Pruning	Structured Pruning
Total Parameters	129,176,036	129,176,036
Non-Zero Parameters	96,339,813	97,088,709
Zero Parameters	32,836,223	32,087,327
Overall Sparsity	25.42%	24.84%
Baseline Model Accuracy	72.11%	72.11%
Accuracy Before Fine-tuning	70.64%	44.28%
Accuracy After Fine-tuning	71.69%	69.24%

Table 1. Accuracy Evaluation Before and After Fine-tuning with 25% Sparsity Induced for Unstructured and Structured Pruning

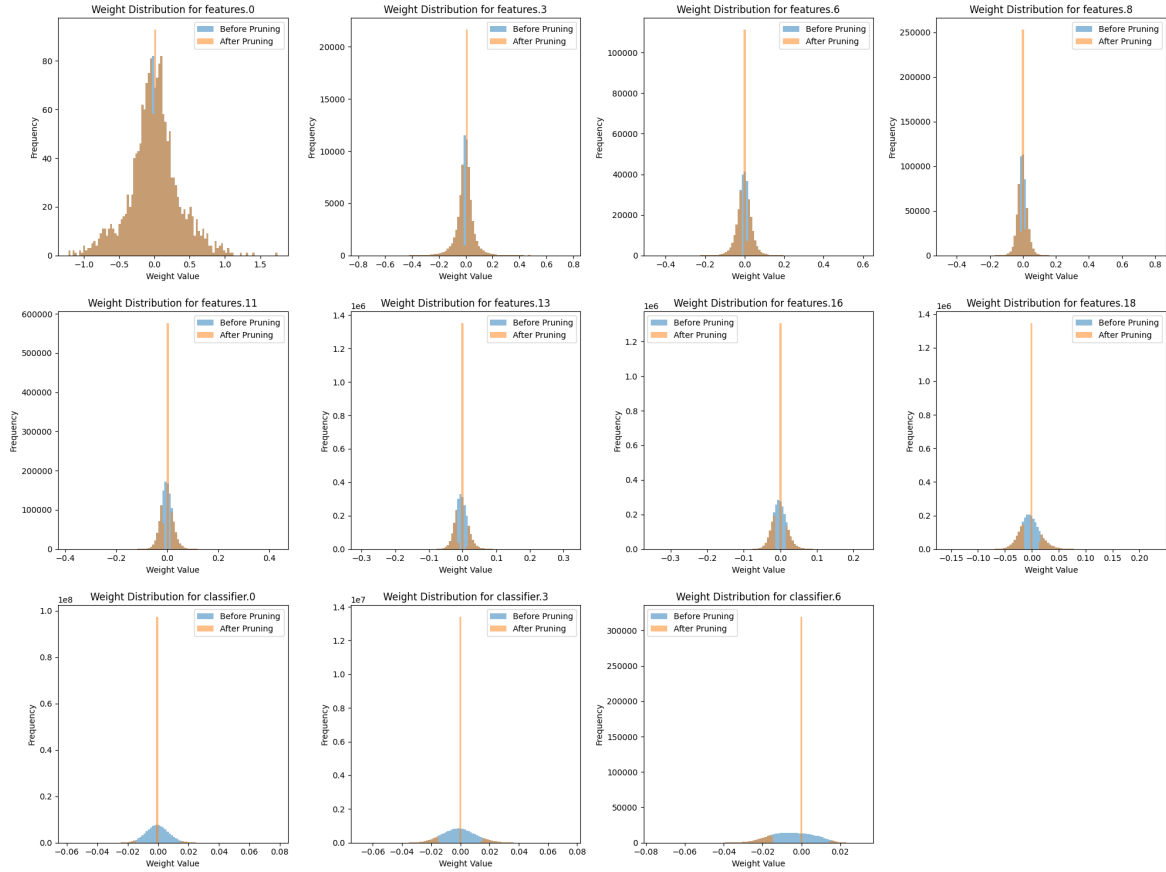


Figure 2. Weight Distribution for Each Layer in VGG-11 Before and After Unstructured Pruning

3.1.2. COMPARITIVE ANALYSIS

Grad-CAM Analysis

The Grad-CAM analysis was conducted to visually assess the impact of pruning on the feature localization capabilities of the VGG-11 model across three configurations: baseline, unstructured pruned, and structured pruned. The input image used for this analysis is shown in Figure 6, while the Grad-CAM visualizations for each model are presented in Figure 7.



Figure 6. Input image used in Grad-CAM analysis.

Baseline Model: The baseline model effectively focuses on

the primary and secondary horse in the image, as indicated by the concentrated heatmap over the animals and the probability of the highest predicted classes. This suggests that the model accurately localizes the region of interest.

Unstructured Pruned Model: Similar to the baseline, the heatmap is concentrated over both the horses, indicating that the model retains its ability to localize key features despite pruning.

Structured Pruned Model: In contrast, the structured pruned model reveals a less defined heatmap over both the horses, suggesting that structured pruning may reduce the model's ability to localize specific features effectively.

The quantitative impact of pruning on classification confidence is reflected in the top-3 predicted class probabilities for each model, as shown below:

- **Baseline Model:**

- Class ID: 39, Probability: 0.1104
- Class ID: 33, Probability: 0.1085
- Class ID: 76, Probability: 0.0823

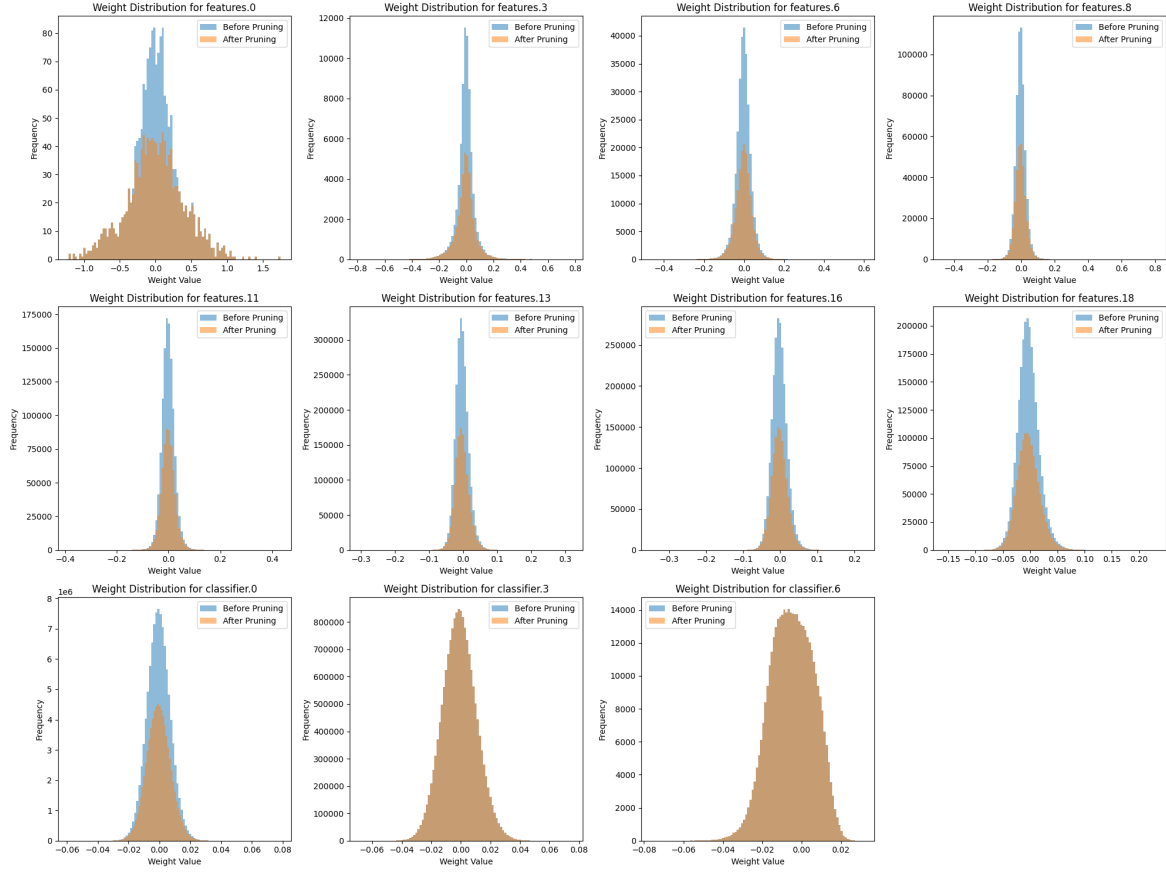


Figure 3. Weight Distribution for Each Layer in VGG-11 Before and After Structured Pruning

• **Unstructured Pruned Model:**

- Class ID: 39, Probability: 0.2419
- Class ID: 13, Probability: 0.0997
- Class ID: 81, Probability: 0.0924

• **Structured Pruned Model:**

- Class ID: 33, Probability: 0.1224
- Class ID: 56, Probability: 0.0792
- Class ID: 81, Probability: 0.0764

Inference Time and Storage Requirements To evaluate the practical impact of pruning, we measured the inference time and storage requirements for the baseline, unstructured pruned, and structured pruned VGG-11 models. The results, summarized in Table 2, show that the structured pruned model achieves the fastest inference time of 4.25 ms, benefiting from the removal of entire channels, which aligns better with hardware acceleration. In contrast, unstructured pruning does not significantly improve inference time due to the model retaining the same size as the baseline. Although unstructured pruning effectively reduces the number of non-zero parameters, it only zeroes out weights without

physically removing them, resulting in minimal impact on inference time.

The storage requirements, also shown in Table 2, indicate that only the structured pruned model achieves a reduction in storage size, shrinking from 492.77 MB to 370.35 MB. This reduction results from the elimination of entire channels, which reduces the overall number of parameters in a way that can be efficiently stored. Conversely, the unstructured pruned model retains the same storage size as the baseline due to the scattered nature of the zero weights, which does not reduce the storage footprint.

Metric	Baseline	Unstructured Pruned	Structured Pruned
Inference Time (ms)	5.39	5.31	4.25
Model Size Before Pruning (MB)	492.77	492.77	492.77
Model Size After Pruning (MB)	492.77	492.77	370.35

Table 2. Comparison of Inference Time and Storage Requirements for Baseline, Unstructured Pruned, and Structured Pruned Models

Accuracy vs. Target Sparsity

The impact of increasing sparsity levels on model accuracy

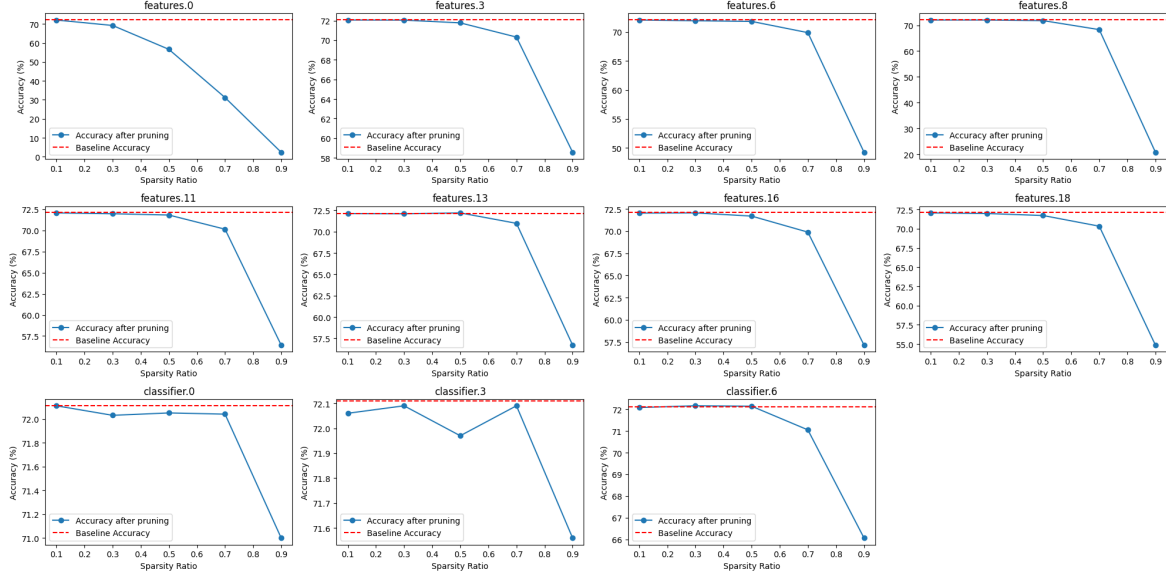


Figure 4. Sensitivity Analysis of Each Layer in VGG-11 at Different Sparsity Ratios (Unstructured Pruning)

for both unstructured and structured pruning methods is displayed in Figure 8. The graph demonstrates how each pruning approach affects the model’s ability to retain performance as the pruning severity increases.

For unstructured pruning, the model maintains a relatively high accuracy across a wide range of sparsity levels. Up to around 60% sparsity, the accuracy remains close to the baseline, indicating that the unstructured pruning method effectively reduces parameter counts without severely impacting performance. This resilience can be attributed to the selective zeroing of individual weights across the layers, which allows the model to maintain critical features for accurate predictions. However, at higher sparsity levels, such as 80%, a more noticeable drop in accuracy is observed, suggesting that unstructured pruning becomes less effective when too many weights are removed.

In contrast, structured pruning, which removes entire channels rather than individual weights, shows a more pronounced decline in accuracy as sparsity increases. At around 40% sparsity, a noticeable reduction in performance begins, and the trend continues sharply downwards as sparsity reaches 80%. This decline reflects the impact of structured pruning’s channel-level removals, which can limit the model’s ability to learn complex patterns due to the reduced representational capacity. Structured pruning can lead to more efficient hardware utilization, but it significantly affects model accuracy at high sparsity levels, indicating a trade-off between computational efficiency and predictive performance.

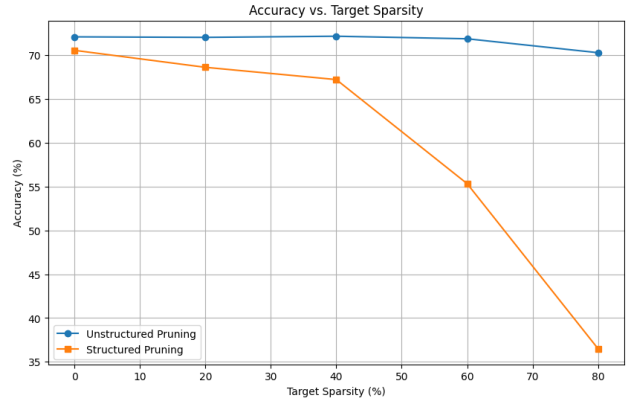


Figure 8. Accuracy vs. Target Sparsity for Unstructured and Structured Pruning

3.2. Quantization

Figure 9 presents the results of quantization schemes against accuracies. It was noted that while PTQ methods, particularly for int4, resulted in slight accuracy degradation (ranging from 68.38% to 68.58%), the application of QAT improved the performance across all configurations. Notably, the int8 configuration under QAT achieved an accuracy of 70.53%, which not only outperforms its PTQ counterpart but also exceeds the baseline accuracy of 68.61%. This demonstrates that QAT allows the model to adapt to the constraints imposed by low-bit-width quantization and recover much of the accuracy lost during PTQ. Also, because the accuracy degradation of PTQ was so marginal that it turns out to be a very viable option for this scenario.

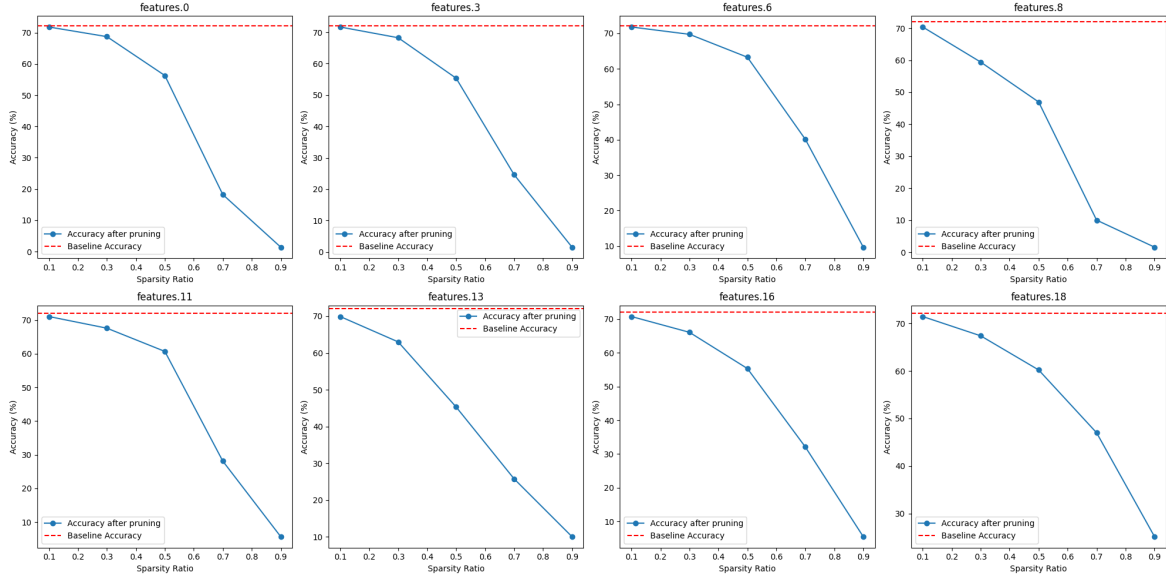


Figure 5. Sensitivity Analysis of Convolutional Layers in VGG-11 at Different Sparsity Ratios (Structured Pruning)

Figure 10 shows that after the application of PTQ, the model’s accuracy slightly drops, but after QAT, the accuracy rises again, even exceeding the baseline. This pattern is consistent across all configurations, indicating that QAT is an effective approach to mitigate the accuracy loss caused by quantization. For example, the int4 model, which showed a notable dip in accuracy with PTQ (68.38%), recovered to 69.02% with QAT, demonstrating that the model benefits from retraining under quantization constraints.

Table 3 also includes the memory footprints of the quantized models against their accuracies. The baseline model, using the standard float32 precision, occupies 492.77 MB of memory. Post-quantization, the memory usage is significantly reduced across all quantization formats. The most notable memory reduction is seen in the int4 quantization, which reduces the model size to just 74.79 MB, reflecting an impressive 85% reduction from the baseline. Both fp16 and bf16 result in 50% reduction in model size, making them viable options for memory-constrained environments.

3.3. Knowledge Distillation

3.3.1. ASPECTS OF LOGIT MATCHING

The accuracies of each model, as shown in Table 4, indicate that Label Smoothing Regularization achieved the highest accuracy. This result suggests that regularizing labels helps the student model generalize better by reducing overconfidence in predictions, which is particularly beneficial in knowledge distillation (KD) setups. Basic Logit Matching also provided a notable performance increase, as it directly aligns the student’s logits with the teacher’s outputs, im-

Dtype	Size (MB)	Base/PTQ/QAT	Accuracy
float32	492.77	Baseline	68.61
float16	246.15	4*PTQ	68.58
bfloat16	246.15		68.52
int8	149.57		68.53
int4	74.79		68.38
float16	246.15	4*QAT	69.98
bfloat16	246.15		69.02
int8	149.57		70.53
int4	74.79		69.02

Table 3. Model Sizes and Accuracies for Different Dtypes and Quantization Methods

proving the student’s ability to mimic the teacher. However, Decoupled Knowledge Distillation (DKD) showed a slightly lower accuracy, likely due to weaker guidance without direct alignment on logits, indicating that direct methods like logit matching and label smoothing are more effective for this dataset.

3.3.2. COMPARING PERFORMANCE OF SOTA APPROACHES

The accuracies of each model are shown in Table 5. These findings suggest that the Hints method provides the highest accuracy among student models, nearly matching the teacher’s performance, while the Logit Matching method also achieves substantial knowledge transfer. The CRD approach, however, showed relatively lower accuracy, indicating that it may be less effective in capturing and transferring



Figure 6. (a) Baseline Model

Figure 6. (b) Unstructured Pruned Model

Figure 6. (c) Structured Pruned Model

Figure 7. Grad-CAM visualizations for (a) Baseline Model, (b) Unstructured Pruned Model, and (c) Structured Pruned Model.

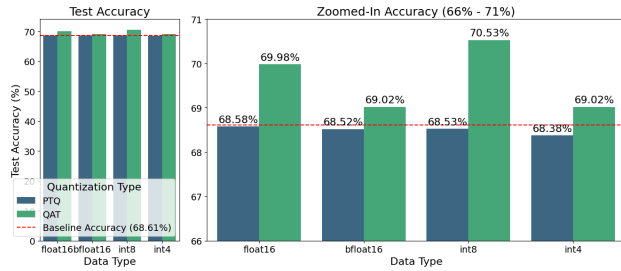


Figure 9. Accuracies against different quantization schemes

Model	Accuracy
Teacher	71.43%
Basic Logit Matching	70.57%
Label Smoothing Regularization	70.62%
Decoupled KD	66.72%

Table 4. Accuracy of Each Model based on Logit Matching

the teacher’s representations to the student model. This comparison highlights the effectiveness of Hints and Logit Matching in preserving the teacher’s learned knowledge within the student model.

3.3.3. COMPARING PROBABILITY DISTRIBUTIONS

The KL divergence values highlight how well each student model aligns with the teacher model’s probability distributions. The KL divergence values for each student model are as follows:

- **Independent Student:** 0.1895

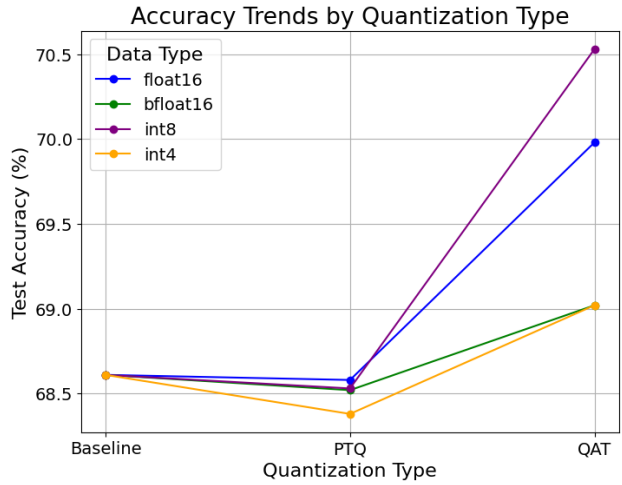


Figure 10. Accuracies for PTQ and QAT

- **Logit-Matching Student:** 0.0749
- **Hint-based Student:** 0.1647
- **CRD Student:** 0.1653

These results indicate that the Logit-Matching student model shows the closest alignment with the teacher, achieving the lowest KL divergence. This suggests that Logit-Matching is particularly effective in bringing the student model’s predictions in line with those of the teacher. In contrast, the independent student model had the highest KL divergence, demonstrating the largest discrepancy in probability distributions and reinforcing the value of knowledge distillation techniques for improving alignment with the teacher. Figure 11 visually illustrates the similarity between the teacher’s and Logit-Matching student’s distributions, further support-

Model	Accuracy
Teacher	71.43%
Independent Student	69.60%
Logit Matching	70.45%
Hints	70.95%
CRD	68.77%

Table 5. Accuracy of Models with Different Knowledge Distillation (KD) Methods

ing this conclusion.

3.3.4. EXAMINING LOCALIZATION KNOWLEDGE TRANSFER

By visualizing the attention maps using Grad-CAM (see Figure 12), we observed that student models trained with the teacher in all three distillation techniques—Logit Matching, Hint-based Distillation, and Contrastive Representation Distillation—focused on similar regions of the image as the teacher model. In contrast, the independent student, trained without any knowledge distillation guidance, exhibited attention focused on different parts of the image, suggesting a divergence in localization knowledge.

To further quantify the alignment between the teacher and the student models in terms of localization, we computed the Intersection over Union (IoU) between the teacher’s attention map and each student model’s attention map for Image 0. The IoU values are as follows:

- IoU between Teacher and Logit Matching: 0.3103
- IoU between Teacher and Hint-based: 0.5000
- IoU between Teacher and CRD: 0.3333
- IoU between Teacher and Independent: 0.2424

These values suggest that the student models trained with distillation techniques achieved a higher overlap with the teacher’s attention, with the Hint-based Distillation model achieving the highest IoU of 0.5000. In contrast, the independent student, with an IoU of 0.2424, showed the least overlap with the teacher’s attention map, further supporting the effectiveness of knowledge distillation in transferring not only class probabilities but also spatial attention patterns.

This alignment of attention maps between the KD-trained student models and the teacher indicates that knowledge distillation effectively transfers both class-representative features and spatial structures within images, allowing students

to replicate the teacher’s focus on relevant image regions. These observations are consistent with the findings in (Ojha et al., 2023), which highlight that KD methods help students capture both class-representative features and spatial structures, thus enhancing interpretability and alignment with the teacher’s reasoning process.

3.3.5. CHECKING FOR COLOR INVARIANCE WITH CRD

In examining color invariance with Contrastive Representation Distillation (CRD), we found that the student model effectively learned to maintain similar accuracy levels even when tested on a color-jittered dataset. Interestingly, the student trained via CRD retained a degree of color invariance, which allowed it to perform better than the independent student model on color-altered images. The independent student achieved the accuracy of 63.78% on the colour jittered test set while it was trained on non-augmented train set. Whereas the student trained via CRD achieved 65

This finding is somewhat unexpected, given that the teacher model’s invariance properties are not explicitly exposed to the student during distillation. During training, the student is tasked only with matching the teacher’s feature representations for the original images, such as X , and does not receive augmented versions (e.g., color-jittered X') of these images. Therefore, the student has no direct exposure to the teacher’s response to altered versions of the input images. Yet, CRD enables the student to approximate this invariance, indicating that certain robust features—such as color invariance—can be implicitly transferred from teacher to student during distillation. This aligns with the surprising findings in (Ojha et al., 2023), which suggest that CRD may allow students to generalize beyond the training constraints imposed.

3.3.6. TESTING THE EFFICACY OF A LARGER TEACHER

With a larger teacher model, the accuracy of the student model increased slightly, from 70.45% to 71.16%, compared to training with a smaller teacher. This improvement can be attributed to the richer and more refined representations that a larger teacher can provide, offering a more detailed probability distribution for the student to learn from. However, as noted in (Cho & Hariharan, 2019), the accuracy of the student model is not heavily dependent on the size of the teacher. The paper suggests that other factors, such as the alignment of intermediate representations and the specific knowledge distillation (KD) technique applied, play a more crucial role in determining the student model’s final performance.

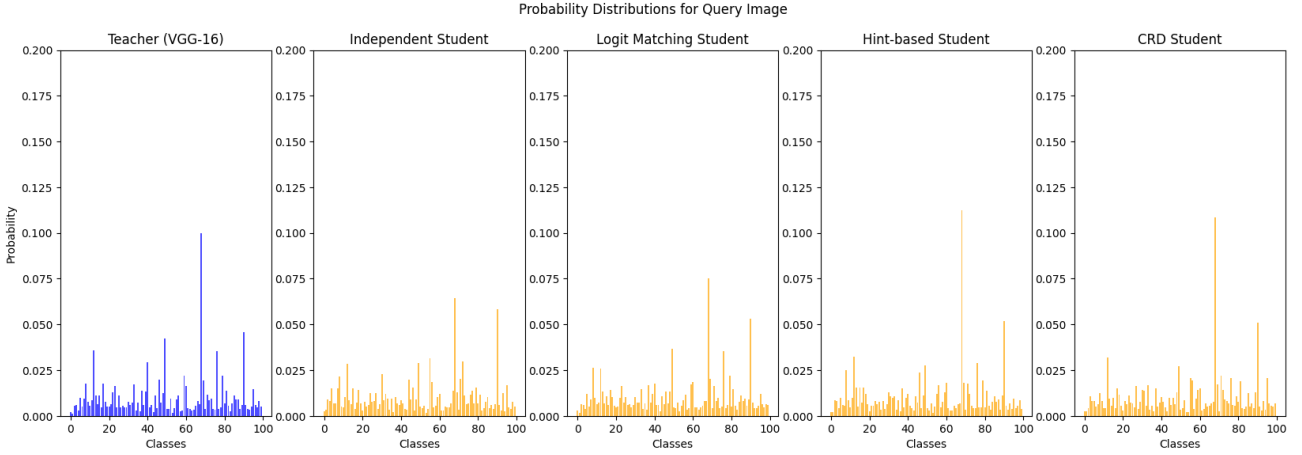


Figure 11. Comparison of Probability Distributions between Teacher and Logit-Matching Student Model. The Logit-Matching student shows a closer alignment with the teacher’s distribution than other distillation methods and the independent student model.

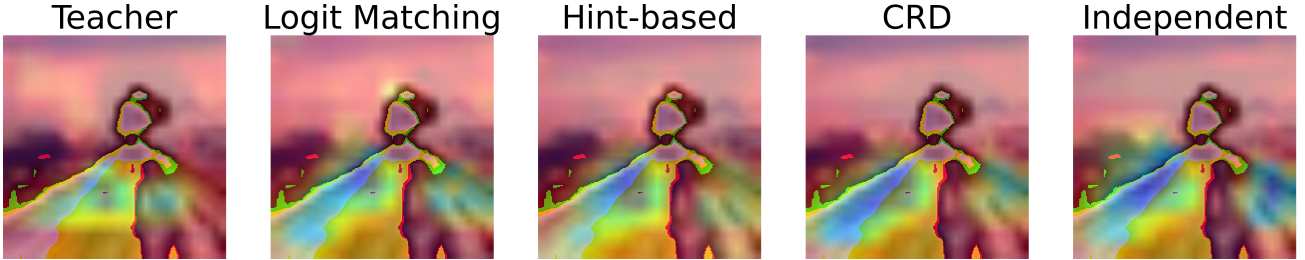


Figure 12. Comparison of Attention maps generated via GradCam

4. Discussion

4.1. Pruning

The results from the pruning experiments demonstrate the distinct behaviors and trade-offs associated with both unstructured and structured pruning approaches on the VGG-11 model. Each approach affects the model’s accuracy, inference time, storage requirements, and feature localization differently, and each carries specific hardware implications.

Both unstructured and structured pruning successfully reduce the number of parameters in the model; however, they impact model accuracy and feature localization differently. Unstructured pruning, which zeroes out individual weights across all layers, achieved minimal accuracy loss and maintained close alignment with the baseline model in Grad-CAM visualizations, suggesting that unstructured pruning is capable of retaining detailed feature representations as can also be seen in the top 1 probability predicted by both models is the same class. In contrast, structured pruning, which removes entire channels, exhibited a more pronounced accuracy drop and a noticeable reduction in focus on relevant features in Grad-CAM. This behavior is particularly evident in early layers that capture foundational patterns, where

channel removal can lead to significant information loss. The structured pruned model also demonstrates a degradation in its ability to attend to precise features within the image, highlighting its inherent limitation in retaining fine-grained details when channels are removed.

In terms of practical efficiency, structured pruning proves advantageous. Structured pruning led to a substantial reduction in model size (from 492.77 MB to 370.35 MB) and the lowest inference time (4.25 ms) due to the removal of entire channels, which inherently reduces the computational workload and aligns better with hardware acceleration. Unstructured pruning, while achieving sparsity, did not reduce the storage size due to scattered zero weights, which still require memory allocation. Similarly, inference time remained closer to that of the baseline model since zeroed weights do not inherently reduce the number of operations required, making it less effective in scenarios where memory and compute resources are limited.

Sensitivity analysis further highlights the differential impact on layers. Both approaches reveal that early layers are more sensitive to pruning due to their role in capturing essential, low-level features. Structured pruning particularly exacerbates this sensitivity, as removing entire channels from these

layers directly impacts the model’s ability to encode basic patterns. Unstructured pruning, however, allows for a more granular reduction in these sensitive layers by selectively removing only unimportant weights. Thus, unstructured pruning offers a finer level of control for balancing accuracy retention with sparsity in sensitive layers.

From a hardware perspective, structured pruning is generally more favorable due to its compatibility with hardware acceleration frameworks, which can exploit the channel-level reduction for faster computation. Many modern accelerators, including GPUs and specialized AI processors, are optimized to handle structured sparsity patterns efficiently, leading to real gains in both speed and memory savings. Conversely, unstructured pruning introduces irregular sparsity, which is challenging for hardware to optimize. Although unstructured pruning can achieve high sparsity levels, the random distribution of zero weights does not leverage the parallel processing capabilities of most hardware, limiting its impact on inference speed and storage reduction. As a result, unstructured pruning may be less suited for deployment on resource-constrained devices.

Overall, structured pruning provides more tangible benefits for real-time applications with constrained resources, owing to its computational efficiency and storage reduction. However, it may suffer in terms of accuracy and feature localization, especially in complex models where channel integrity is critical. Unstructured pruning, while achieving higher sparsity without heavily sacrificing accuracy, lacks the same level of efficiency gains in deployment, making it more suitable for scenarios where accuracy is prioritized over speed and storage.

In conclusion, the choice between structured and unstructured pruning depends on the application requirements. Structured pruning is advantageous for deployment on hardware that can leverage channel-level sparsity, while unstructured pruning is preferable for applications needing high accuracy with minimal feature loss.

4.2. Quantization

The results of this study reveal several important insights into the trade-offs between quantization, memory efficiency, and accuracy.

While PTQ provides a faster and computationally cheaper solution, its performance often suffers, especially when quantizing to lower bit-widths like int4. In contrast, QAT significantly improves the performance of the model, especially for aggressive quantization like int8 and int4. Although QAT requires more computational resources and time, it is essential for achieving high accuracy when quantizing models for deployment.

The issues encountered with CUDA support for int4 high-

light the importance of considering hardware limitations when choosing the appropriate quantization format. While floating-point formats (fp16 and bf16) are widely supported and stable, integer formats like int8 and especially int4 might not be universally supported across all GPU architectures.

Furthermore, the results clearly indicate that lower bit-widths, such as int8 and int4, offer significant memory savings. However, these savings come at the slight cost of model accuracy unless QAT is employed. The choice of bit-width should be based on the specific requirements of the deployment scenario—if memory is at a premium and slight accuracy degradation is acceptable, int4 might be a good choice. However, for applications that require the highest accuracy, QAT with higher bit-width should be prioritized. These findings align with the work by Dettmers and Zettlemoyer (Dettmers & Zettlemoyer, 2023), where maintaining a constant total model bit count while increasing the number of parameters or layers led to higher accuracy for lower bit-widths. Our observations echo this trend: a quantized model with approximately 15% of the memory footprint of the baseline model retains nearly equivalent accuracy, underscoring the effectiveness of quantization, particularly when combined with QAT.

The comparison between fp16 and bf16 was also critical in understanding how different floating-point formats impact the stability of training. While both formats are 16-bit and achieve same memory reductions, their internal structure differs significantly: fp16 uses 1 sign bit, 5 exponent bits, and 10 mantissa bits, while bf16 uses 1 sign bit, 8 exponent bits, and 7 mantissa bits. The extra exponent bits in bf16 extend its dynamic range, making it better suited for representing both very large and very small values—a crucial advantage in deep networks where gradient values vary widely across layers. In our experimentation, bf16 consistently offered more stable training than fp16, with the latter often encountering NaN losses during QAT. This is because fp16’s limited exponent range makes it more prone to issues like underflow and overflow of gradients. For deep learning tasks requiring stable training and precise gradient handling, bf16 clearly emerges as the preferable format.

4.3. Knowledge Distillation

Our experiments with logit matching, label smoothing regularization (LSR), and decoupled knowledge distillation (DKD) show that **LSR** yielded the highest accuracy. LSR improves the student’s generalization by preventing overconfidence, effectively softening the target labels. This helps the model avoid overfitting by smoothing the class distributions, a crucial aspect in distillation tasks where the teacher model’s knowledge is transferred to the student. The improved robustness observed is consistent with prior work,

which has shown that regularizing the labels in this way reduces the effect of noisy labels.

Logit Matching also boosted performance by directly aligning the logits of the student model with those of the teacher. This alignment encourages the student to closely mimic the teacher’s decision-making process, leading to improved accuracy. However, **DKD** showed slightly lower performance. This could be due to DKD’s more indirect approach to transferring knowledge, where the lack of direct alignment between teacher and student may weaken the guidance, especially in cases where the teacher-student gap is large or training epochs are limited. Thus, DKD may require more epochs or a better teacher-student alignment for optimal performance.

Among state-of-the-art (SOTA) methods, **Hints** was the most effective, achieving accuracy very close to the teacher’s. The Hints method works by transferring the teacher’s intermediate feature representations to the student, providing fine-grained guidance for the student to learn. This method ensures that the student can capture not just the final decision but also the internal representations that lead to that decision, resulting in more accurate knowledge transfer. **Logit Matching** was effective, though slightly less so than Hints. This can be attributed to Logit Matching’s focus on aligning the final output logits, which, while helpful, may not fully capture the depth of the teacher’s internal knowledge. In contrast, **Contrastive Representation Distillation (CRD)** performed poorly, likely because it focuses on aligning representations in an unsupervised manner, which may be less effective at directly mimicking the teacher’s learned patterns compared to methods like Logit Matching and Hints, which align logits and intermediate representations more directly.

KL divergence further supported the effectiveness of Logit Matching. The Logit-Matching student model achieved the lowest divergence, indicating that it closely matched the teacher’s probability distribution. This suggests that direct alignment on logits is highly effective in guiding the student to make more accurate predictions, as it forces the student to mirror the teacher’s confidence levels. On the other hand, the independent student, with no distillation guidance, exhibited the highest KL divergence, highlighting the benefit of distillation methods in improving alignment and reducing discrepancies between the teacher’s and student’s predictions.

Grad-CAM visualizations showed that knowledge distillation techniques successfully transferred localization knowledge. The student models trained with knowledge distillation (Logit Matching, Hint-based, and CRD) focused on similar image regions as the teacher, suggesting that they learned the teacher’s spatial attention patterns. This is crucial for tasks that require both accurate classification and

precise localization, such as object detection. In contrast, the independent student, not trained with distillation, focused on different regions, showing that distillation encourages better localization transfer. Among the distillation methods, **Hint-based Distillation** achieved the highest *Intersection over Union (IoU)* with the teacher’s attention map, suggesting that Hints transfers both class-representative features and spatial structures more effectively than other methods.

CRD demonstrated unexpected robustness to color variations. Despite not being trained explicitly on color-altered images, the CRD student performed better than the independent student on color-jittered images. This suggests that CRD is effective at learning invariant features that generalize across color perturbations, likely due to its focus on contrasting representations in a way that emphasizes robust features. This aligns with prior studies suggesting that CRD can facilitate the transfer of robust, generalizable features, even if they are not directly present in the training data.

Finally, testing the effect of a **larger teacher model** led to a slight increase in student accuracy (from 70.45% to 71.16%). This improvement is likely due to the more refined and detailed feature representations that a larger teacher model provides. Larger models tend to have better capacity to capture complex patterns, which can help the student model learn more accurate representations. However, as suggested by Cho and Hariharan (2019), the teacher model’s size is not the most important factor influencing student performance. Instead, the alignment of intermediate representations and the specific distillation method used are the critical factors. While a larger teacher can offer richer knowledge, the method of transferring this knowledge (e.g., through logits or intermediate representations) plays a more pivotal role in shaping the student’s final performance.

4.4. Comparison of Compression Approaches

In this section, we present a holistic comparison of three model compression techniques—Pruning, Quantization, and Knowledge Distillation (KD)—focusing on their practical convenience, advantages and drawbacks, effectiveness under resource constraints, and potential synergies when used together. Each technique brings distinct benefits and trade-offs, which impact its optimal deployment context.

4.4.1. CONVENIENCE AND PRACTICALITY

Each compression technique has unique characteristics influencing its ease of implementation and practicality in real-world scenarios. Pruning can be applied in either unstructured or structured forms. Unstructured pruning removes individual weights deemed less important, offering a fine level of control over sparsity as can be seen from the sensitivity analysis Figure 4. Structured pruning, on the other hand, removes entire channels, making it more compatible

with hardware that supports structured sparsity. This type of pruning is easier to deploy on hardware but can lead to noticeable accuracy drops if critical channels are removed, especially in the earlier layers of the model that capture foundational features.

Quantization, particularly in its Post-Training Quantization (PTQ) form, is relatively straightforward and computationally inexpensive, making it practical for quick compression needs. However, PTQ often struggles to retain model accuracy at very low bit-widths (e.g., INT4). Quantization-Aware Training (QAT) provides a more robust solution by incorporating quantization during training, which enables the model to retain higher accuracy even at aggressive bit-widths. However, QAT requires greater computational resources and extensive parameter tuning, particularly when targeting low bit-widths across large models.

Knowledge Distillation (KD) offers a different form of model compression by training a smaller student model to mimic the outputs of a larger, pre-trained teacher model. KD is conceptually simple, making it a flexible approach to achieve model simplification. However, achieving optimal results may require additional tuning, particularly when the student and teacher architectures differ significantly or when multi-stage distillation is employed. While straightforward in principle, selecting the optimal distillation loss or balancing between feature representation and output matching can add complexity to KD implementations.

In summary, PTQ emerges as the most accessible and practical method when accuracy is not a primary concern. Both QAT and KD, while more computationally intensive, offer greater accuracy retention and are better suited for scenarios that require high-performance, compact models. The choice of technique depends on the desired balance between simplicity and accuracy preservation.

4.4.2. POTENTIAL DRAWBACKS AND ADVANTAGES

Each compression technique presents unique advantages and potential limitations. Pruning, particularly unstructured pruning, provides fine-grained control over the model's sparsity, allowing it to retain important feature representations without heavily impacting accuracy. Structured pruning, although advantageous for hardware optimization, may degrade accuracy by removing entire channels, leading to potential information loss and reduced model robustness, especially in early layers as can be seen in sensitivity analysis Figure 5 of layers during structured pruning. Moreover, unstructured pruning does not inherently reduce computational requirements on hardware that lacks support for irregular sparsity, thus limiting its efficiency benefits in deployment as discussed in results 4.4.3.

Quantization, especially through QAT, enables substantial

memory and computational savings, which aligns well with hardware configurations supporting low-bit integer formats like INT8. This approach preserves accuracy effectively, even at aggressive compression levels. However, PTQ may struggle to retain model accuracy at very low bit-widths, and not all hardware platforms support low-bit formats such as INT4, particularly for floating-point operations, where issues such as NaN errors can arise with formats like FP16 during training.

Knowledge Distillation is highly advantageous for reducing model size while preserving performance levels comparable to the teacher model. This technique also simplifies the model for faster inference, making it well-suited for constrained environments. However, KD has limitations, particularly in cases where the student model inherits biases or vulnerabilities from the teacher, such as susceptibility to adversarial attacks. As we saw that colour invariance of the teacher was also being passed implicitly to the students. Given this, we have lesser control on what to pass to the student and what not to. Additionally, KD may not perform optimally when the teacher and student architectures are substantially different, as it becomes challenging for the student to fully capture the knowledge embedded in a larger model.

In summary, structured pruning is well-suited for hardware-optimized applications where inference efficiency is prioritized. QAT-based quantization excels in scenarios where memory savings are critical but accuracy retention is necessary. KD offers a balanced approach for cases requiring a smaller, more interpretable model, though attention should be given to the potential transfer of the teacher model's limitations.

4.4.3. EFFECTIVENESS GIVEN A FIXED COMPUTE BUDGET

Under a fixed compute budget, each technique offers different trade-offs in terms of simplicity, accuracy, and efficiency. Pruning, especially structured pruning, achieves efficient resource use with minimal computational demands during both training and inference, making it well-suited for deployment scenarios with limited compute resources. Unstructured pruning, while beneficial for maintaining accuracy, may not significantly reduce computational load on hardware that does not support irregular sparsity patterns. All this can be seen from the storage requirements and inference speeds in results

Quantization provides considerable memory and computational savings (as can be seen in Table 3), particularly when using PTQ for rapid deployment. However, QAT may be challenging to implement under strict resource constraints due to the increased computational demand of training a model with quantization in the loop. For applications where

memory reduction is essential, PTQ is highly efficient; for higher accuracy retention at low bit-widths, QAT is more appropriate if additional compute resources are available.

Knowledge Distillation, although computationally intensive during training, is highly efficient in inference, as it enables deployment of a smaller student model. Simpler KD methods like Logit Matching can be applied under limited compute budgets, offering reasonable accuracy gains without the complexity of advanced KD techniques.

Overall, structured pruning and PTQ are the most feasible options under strict resource constraints, while KD and QAT offer higher accuracy potential when the compute budget allows for additional tuning and training.

4.4.4. COMBINING COMPRESSION APPROACHES

Combining multiple compression techniques can lead to further efficiency gains by leveraging their complementary strengths. For instance, combining pruning with quantization—specifically structured pruning with QAT—can yield a model that is both compact and accurate. While pruning reduces the model’s parameter count, quantization compresses memory usage further without substantial accuracy loss. This combination can be particularly effective for models deployed on hardware optimized for structured sparsity and low-bit arithmetic.

Knowledge Distillation also complements pruning and quantization well. By transferring knowledge from a larger teacher model, KD can enhance the performance of a pruned or quantized student model, helping to retain critical information while ensuring the student model’s robustness. This synergy is especially useful when mild pruning or quantization is applied, as it helps the student model approximate the performance of the teacher with fewer resources.

However, combining multiple techniques requires careful balancing. Excessive pruning or aggressive quantization may degrade model performance to the extent that KD cannot fully recover the accuracy loss. Therefore, a moderate application of each method is recommended to achieve an optimal balance between efficiency and accuracy.

4.4.5. FINAL RECOMMENDATION

For a balanced approach that maximizes memory savings, accuracy retention, and efficiency, Quantization-Aware Training (QAT) emerges as the most effective standalone technique. It enables aggressive compression with minimal accuracy loss, making it suitable for applications where memory efficiency is paramount. For deployment on resource-constrained devices, structured pruning and PTQ offer efficient compression while maintaining reasonable accuracy and computational simplicity.

The combination of Knowledge Distillation with mild pruning or quantization is highly recommended for creating lightweight models that maintain robustness and accuracy. KD enhances the compressed model by retaining essential information, while pruning or quantization optimizes resource efficiency. This combination results in a well-rounded approach for applications that prioritize interpretability, deployment efficiency, and memory savings.

In summary, while each technique offers unique advantages, the combination of KD with structured pruning or QAT provides a holistic solution for model compression. This approach achieves a balanced trade-off between model accuracy, memory usage, and inference speed, aligning well with various deployment requirements and hardware constraints.

5. Conclusion

In conclusion, this study provides a comparative analysis of pruning, quantization, and knowledge distillation as model compression techniques, evaluating their effectiveness in optimizing deep learning models for deployment on resource-constrained platforms. Structured pruning was shown to be advantageous for hardware-accelerated environments, providing significant reductions in model size and inference time, while unstructured pruning maintained better accuracy but with limited efficiency gains on unsupported hardware. Quantization, particularly through Quantization-Aware Training, emerged as the most effective approach for substantial memory savings with minimal accuracy loss, especially in low bit-width configurations. Knowledge distillation allowed for effective model compression by transferring learned representations from larger teacher models to compact student models, offering a balanced trade-off between accuracy and model simplicity.

Each technique demonstrated unique strengths and limitations, highlighting their suitability for specific deployment scenarios. For real-time applications where hardware compatibility is crucial, structured pruning is recommended. QAT-based quantization is ideal for maximizing memory efficiency, while knowledge distillation is beneficial in scenarios that prioritize interpretability and computational simplicity. Overall, these findings underscore the need to align compression strategy selection with deployment requirements, with potential benefits from combining techniques to achieve optimized model performance in diverse applications.

6. Contributions

Author	Tasks Completed
M. Ibrahim Farrukh	Task 1, Task 4
Mustafa Abbas	Task 2, Task 4
Muhammad Safiullah	Task 3, Task 4

Table 6. Contributions of Each Author

References

- Cho, J. H. and Hariharan, B. On the efficacy of knowledge distillation. *arXiv preprint arXiv:1902.05502*, 2019.
- Dettmers, T. and Zettlemoyer, L. The case for 4-bit precision: k-bit inference scaling laws. *arXiv preprint arXiv:2212.09720*, 2023. URL <https://arxiv.org/pdf/2212.09720>.
- Ojha, U., Li, Y., Rajan, A. S., Liang, Y., and Lee, Y. J. What knowledge gets distilled in knowledge distillation? *arXiv preprint arXiv:2302.05392*, 2023.
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. Grad-cam: Visual explanations from deep networks via gradient-based localization. *International Journal of Computer Vision*, 128(2):336–359, 2019. doi: 10.1007/s11263-019-01215-x. URL <https://link.springer.com/article/10.1007/s11263-019-01215-x>.

A. Appendix: Code for Sensitivity Analysis Pruning Function

This section includes the Python code for the `prune_model_per_layer` function, used to perform sensitivity analysis with per-layer pruning and backward hooks.

```
def prune_model_per_layer(model,
    layer_sparsity):
    for name, module in model.named_modules():
        if isinstance(module, (nn.Conv2d, nn.Linear)):
            sparsity = layer_sparsity.get(name, 0)
            if sparsity > 0:
                # Get absolute values of the weights
                weight = module.weight.data.abs()
                flattened_weights = weight.view(-1)
                k = int(len(flattened_weights) * sparsity)
                if k == 0:
                    continue
                # Determine the threshold
                threshold, _ = torch.kthvalue(flattened_weights, k)
                # Create the mask
                mask = weight.ge(threshold).float()
                mask = mask.to(module.weight.device)
                with torch.no_grad():
                    # Apply the mask to the weights
                    module.weight.data.mul_(mask)
                # Store the mask as a buffer
                module.register_buffer('weight_mask', mask)
                # Remove previous hook if any
                if hasattr(module, 'weight_hook_handle'):
                    module.weight_hook_handle.remove()
                # Define a hook function to keep pruned weights at zero
                def backward_hook(grad, module=module):
                    return grad * module.weight_mask
                module.weight_hook_handle = module.weight.register_hook(backward_hook)
            else:
                # Remove existing hook and mask if any
                if hasattr(module, 'weight_hook_handle'):
                    module.weight_hook_handle.remove()
```

```
del module.weight_hook_handle
if hasattr(module, 'weight_mask'):
    del module.weight_mask
```

Listing 1. Function for Per-Layer Pruning with Backward Hooks

B. Appendix: Code for Structured Pruning Function

This section contains the Python code for the `prune_conv_layer_classifier_fix` function, used to perform structured pruning by removing entire channels from convolutional layers and adjusting subsequent layers accordingly.

```
def prune_conv_layer_classifier_fix(model,
    layer_name, num_channels_to_prune):
    module_list = dict(model.named_modules())
    conv = module_list[layer_name]
    next_name = None

    layer_found = False
    for name, module in model.named_modules():
        if layer_found and isinstance(module, nn.Conv2d):
            next_name = name
            break
        if name == layer_name:
            layer_found = True

    importance = compute_channel_importance(conv)
    prune_indices = np.argsort(importance)[:num_channels_to_prune]
    keep_indices = np.argsort(importance)[num_channels_to_prune:]
    new_out_channels = conv.out_channels - num_channels_to_prune
    new_conv = nn.Conv2d(
        in_channels=conv.in_channels,
        out_channels=new_out_channels,
        kernel_size=conv.kernel_size,
        stride=conv.stride,
        padding=conv.padding,
        dilation=conv.dilation,
        groups=conv.groups,
        bias=(conv.bias is not None)
    ).to(conv.weight.device)

    new_conv.weight.data = conv.weight.data[keep_indices, :, :, :].clone()
    if conv.bias is not None:
        new_conv.bias.data = conv.bias.data[keep_indices].clone()

    parent_module = model
    hierarchy = layer_name.split('.')
    for h in hierarchy[:-1]:
```

```

        parent_module = getattr(parent_module
                                , h)
setattr(parent_module, hierarchy[-1],
        new_conv)

if next_name:
    next_conv = module_list[next_name]
    new_in_channels = next_conv.
        in_channels -
        num_channels_to_prune
    new_next_conv = nn.Conv2d(
        in_channels=new_in_channels,
        out_channels=next_conv.
            out_channels,
        kernel_size=next_conv.kernel_size,
        stride=next_conv.stride,
        padding=next_conv.padding,
        dilation=next_conv.dilation,
        groups=next_conv.groups,
        bias=(next_conv.bias is not None)
    ).to(next_conv.weight.device)

    new_next_conv.weight.data = next_conv.
        weight.data[:, keep_indices, :,
        :].clone()
    if next_conv.bias is not None:
        new_next_conv.bias.data =
            next_conv.bias.data.clone()

    parent_module = model
    hierarchy = next_name.split('.')
    for h in hierarchy[:-1]:
        parent_module = getattr(
            parent_module, h)
    setattr(parent_module, hierarchy[-1],
        new_next_conv)
else:
    input_height, input_width = 224, 224
    with torch.no_grad():
        dummy_input = torch.randn(1, 3,
            input_height, input_width).to(
            conv.weight.device)
        output = model.features(
            dummy_input)
        print(f"Output_shape_before_
            flattening:_{output.shape}")
        channels = output.size(1)
        H = output.size(2)
        W = output.size(3)
        HW = H * W
        output = output.view(output.size
            (0), -1)
        new_in_features = output.size(1)

    classifier = model.classifier
    fc = classifier[0]
    new_fc = nn.Linear(
        in_features=new_in_features,
        out_features=fc.out_features,
        bias=(fc.bias is not None)
    ).to(fc.weight.device)

    old_in_features = fc.in_features
    old_channels = conv.out_channels
    old_HW = H * W
    
```

```

    assert old_in_features ==
        old_channels * old_HW, "Mismatch_
        in_old_in_features_calculation"
    new_fc.weight.data.zero_()

    for new_idx, old_channel_idx in
        enumerate(keep_indices):
        old_start_idx = old_channel_idx *
            HW
        old_end_idx = (old_channel_idx +
            1) * HW

        new_start_idx = new_idx * HW
        new_end_idx = (new_idx + 1) * HW

        new_fc.weight.data[:,
            new_start_idx:new_end_idx] =
            fc.weight.data[:,
            old_start_idx:old_end_idx].
            clone()

        if fc.bias is not None:
            new_fc.bias.data = fc.bias.data.
                clone()

    classifier[0] = new_fc
    
```

Listing 2. Function for Structured Pruning of Convolutional Layers

C. Appendix: Grad-CAM Visualization Code

This section contains the Python code for the apply_gradcam function, used to generate Grad-CAM visualizations for the baseline, unstructured-pruned, and structured-pruned models.

```

def apply_gradcam(model, model_name,
    class_idx=None):
    target_layers = [model.features[-3]] #
        Selecting the third last
        convolutional layer
    cam = GradCAM(model=model, target_layers
        =target_layers, use_cuda=torch.cuda.
        is_available())

    # Forward pass
    outputs = model(input_tensor)

    # Compute probabilities
    probabilities = torch.nn.functional.
        softmax(outputs, dim=1)
    top_prob, top_catid = torch.topk(
        probabilities, 5)
    print(f"Top_5_Predictions_for_{
        model_name}:")
    for i in range(top_prob.size(1)):
        class_id = top_catid[0][i].item()
        prob = top_prob[0][i].item()
        print(f"Class_ID:_{class_id},_
            Probability:_{prob:.4f}")

    # Determine the target class
    
```

```
if class_idx is not None:
    targets = [ClassifierOutputTarget(
        class_idx)]
else:
    _, predicted_class = outputs.max(1)
    targets = [ClassifierOutputTarget(
        predicted_class.item())]

# Generate Grad-CAM
grayscale_cam = cam(input_tensor=
    input_tensor, targets=targets)
grayscale_cam = grayscale_cam[0]

# Overlay the Grad-CAM on the original
image
visualization = show_cam_on_image(
    original_image_np, grayscale_cam,
    use_rgb=True)

# Display the visualization
plt.figure(figsize=(8, 8))
plt.imshow(visualization)
plt.axis('off')
plt.title(f'Grad-CAM_{model_name}')
plt.show()
```

Listing 3. Grad-CAM Visualization Implementation